# Consola para Administración de Máquinas Virtuales GNU/Linux

Jorge Acosta Andrey Tombe

Universidad del Quindío
Facultad de Ingeniería
Ingeniería de Sistemas y Computación
Computación en la Nube
Octubre del 2026

***Resumen*—(Redactar al final del informe.)**

***Index Terms*—palabra clave, palabra clave, palabra clave**

## I. Introducción

La administración de sistemas operativos GNU/Linux es una actividad fundamental en el área de computación en la nube. Tareas como la instalación de paquetes, la configuración del servidor SSH, la gestión de servicios y el balanceo de carga se realizan, en muchos entornos, de manera manual y a través de comandos introducidos directamente en una terminal. Este enfoque, si bien es válido, resulta propenso a errores de digitación, difícil de estandarizar y poco accesible para usuarios que no dominan la línea de comandos.

A lo largo del curso de Computación en la Nube se han ejecutado estas operaciones de forma tradicional. En el presente trabajo se propone convertir dichas tareas en operaciones que pueden solicitarse desde una aplicación web desarrollada en el lenguaje Go. La aplicación funciona como una consola de automatización para una máquina virtual que ejecuta GNU/Linux Debian 13, integrando las operaciones del sistema en una interfaz de usuario que permite validar las entradas, ejecutar las acciones de manera no interactiva y reportar el resultado de cada proceso, incluyendo mensajes de error comprensibles y la información técnica necesaria para el diagnóstico.

## II. Marco Teórico

### *II-A. Máquinas virtuales y virtualización*

La virtualización permite ejecutar múltiples sistemas operativos aislados sobre un mismo hardware físico mediante un hipervisor, que intermedia el acceso a los recursos de la máquina. En este trabajo se utiliza Oracle VM VirtualBox, un hipervisor de tipo 2 (alojado sobre un sistema operativo anfitrión) multiplataforma, disponible para GNU/Linux y Windows [1].

VirtualBox se administra tanto desde su interfaz gráfica como desde la utilidad de línea de comandos `VBoxManage`, incluida en toda instalación y disponible de forma idéntica en todos los sistemas operativos soportados [1]. Esta característica es relevante para la consola: cualquier operación que la aplicación deba realizar sobre la máquina virtual (consultar su estado, obtener su dirección IP, ejecutar un programa dentro de ella) puede expresarse como llamadas a `VBoxManage`, lo que la convierte en la base de la comunicación inicial con la máquina virtual.

Las Guest Additions son controladores y servicios que se instalan dentro de la máquina virtual y mejoran la integración con el anfitrión: redimensionado de pantalla, portapapeles compartido, directorios compartidos y, en particular, la publicación de propiedades del sistema invitado [2]. Entre estas propiedades se encuentra la dirección IP de la máquina, publicada como una propiedad consultable desde el anfitrión con el nombre `/VirtualBox/GuestInfo/Net/0/V4/IP`. Este mecanismo resuelve un problema práctico del proyecto: antes de que exista cualquier servicio de red configurado dentro de la máquina (como SSH), la consola necesita conocer su dirección para poder establecer la primera conexión.

### *II-B. Sistema operativo GNU/Linux Debian*

### *II-C. Gestión de paquetes con APT*

### *II-D. Protocolo y servicio SSH*

El protocolo Secure Shell (SSH) proporciona canales seguros sobre redes no confiables, con autenticación cifrada, confidencialidad e integridad de las comunicaciones [3]. En servidores GNU/Linux el servidor de referencia es OpenSSH, que implementa el servicio `sshd` y el cliente homónimo.

SSH admite autenticación por contraseña y por par de llaves criptográficas asimétricas. En el esquema por llaves, el cliente posee una llave privada y el servidor guarda la llave pública correspondiente en el archivo `~/.ssh/authorized_keys` de cada usuario; el servidor verifica la firma del cliente sin transmitir secretos, y es posible deshabilitar por completo el acceso por contraseña [3]. Este es el mecanismo que la consola configura

en la máquina virtual: una vez instalado y habilitado SSH, toda la comunicación posterior entre la consola y la máquina se realiza por este canal, reemplazando la comunicación inicial basada en Guest Additions.

Además de la ejecución interactiva de comandos, el cliente OpenSSH permite ejecutar una orden remota de forma no interactiva (`ssh usuario@host orden`) y transferir archivos mediante `scp`, lo que lo convierte en una interfaz de programación completa para automatizar administración de sistemas [4].

### *II-E. Gestión de servicios con systemd*

### *II-F. Balanceo de carga con HAProxy*

### *II-G. Lenguaje de programación Go y aplicaciones web*

Go es un lenguaje compilado, de tipado estático y concurrencia ligera, diseñado para producir binarios simples y multiplataforma [4]. Compilar el mismo código para GNU/Linux y Windows sin dependencias en tiempo de ejecución encaja con el objetivo de esta consola, ya que la aplicación debe poder ejecutarse en el equipo anfitrión independientemente de su sistema operativo.

La interfaz web se sirve con el paquete estándar `net/http` y sigue una arquitectura cliente-servidor: el navegador presenta los módulos de administración y envía formularios; el servidor en Go valida las entradas, ejecuta las operaciones sobre la máquina virtual y devuelve el resultado.

Para ejecutar operaciones externas, Go ofrece el paquete `os/exec`, que crea y controla procesos del sistema operativo capturando su salida, su error y su código de salida [4]. Este es el primitivo sobre el que se construye la biblioteca de conectores: la consola no invoca `vboxmanage` ni `ssh` directamente desde su lógica, sino a través de procesos externos lanzados de forma controlada, lo que permite usar los mismos binarios en Windows y en GNU/Linux.

### *II-H. Arquitectura de puertos y adaptadores*

Para mantener el núcleo de la aplicación independiente de la infraestructura concreta se aplica el patrón de arquitectura de puertos y adaptadores (también llamado arquitectura hexagonal) [5]. En este patrón, el núcleo define *puertos*: interfaces que expresan lo que la aplicación necesita del exterior (ejecutar un programa, abrir una sesión con la máquina, conocer su dirección). Los *adaptadores* son implementaciones concretas de esos puertos, que traducen el lenguaje de una tecnología específica al lenguaje del dominio.

En este proyecto los puertos son `Executor` (ejecutar un proceso en el anfitrión), `Machine` (obtener dirección y estado de la máquina virtual), `Connector` (establecer sesiones) y `Session` (ejecutar órdenes y transferir archivos dentro de la máquina). Los adaptadores implementados son `localproc` (procesos del sistema operativo), `vbox` (VirtualBox mediante `vboxmanage` y Guest Additions) y `ssh` (cliente OpenSSH). De este modo, el núcleo no sabe que se usa Guest Additions ni VirtualBox: solo conoce interfaces, y cambiar una tecnología por otra (u otra distinta) no afecta a la lógica de la consola.

Figura 1. Escenario utilizado para desarrollar y probar la solución.

## III. Diseño de la Solución

### *III-A. Escenario de Aplicación*

#### *III-A1. Descripción del Escenario:*

#### *III-A2. Mecanismo de Comunicación:*

#### *III-A3. Flujo General de la Automatización:*

### *III-B. Arquitectura de la Solución*

#### *III-B1. Visión General de la Aplicación:*

#### *III-B2. Estructura del Proyecto:*

#### *III-B3. Decisiones Tomadas Durante el Desarrollo:*

## IV. Implementación

### *IV-A. Preparación del Sistema*

#### *IV-A1. Instalación y Eliminación de Aplicaciones:*

#### *IV-A2. Configuración de la Llave de Acceso SSH para el Usuario Root:*

#### *IV-A3. Personalización de la Entrada del Sistema:*

#### *IV-A4. Asignación del Nombre de Host y Actualización del Archivo de Hosts:*

### *IV-B. Gestión de Usuarios*

#### *IV-B1. Creación de Usuarios y Privilegios de sudo:*

#### *IV-B2. Eliminación de Usuarios:*

### *IV-C. Gestión de Demonios con systemd*

### *IV-D. Gestión de un Balanceador de Carga con HAProxy*

## V. Pruebas

### *V-A. Diseño de las Pruebas*

### *V-B. Resultados y Evidencias*

## VI. Conclusiones

### *VI-A. Problemas Encontrados y Soluciones*

### *VI-B. Herramientas de Inteligencia Artificial Utilizadas*

### *VI-C. Conclusiones del Trabajo*

## Referencias


[1] *Oracle VM VirtualBox User Manual*, 7.1, Oracle, 2025. dirección: https://www.virtualbox.org/manual/

[2] *VirtualBox Guest Additions*, Oracle, 2025. dirección: https://www.virtualbox.org/manual/ch04.html

[3] T. Ylonen y C. Lonvick, *The Secure Shell (SSH) Authentication Protocol*, RFC 4252, Internet Engineering Task Force, 2006. dirección: https://www.rfc-editor.org/rfc/rfc4252

[4] A. A. A. Donovan y B. W. Kernighan, *The Go Programming Language*. Boston: Addison-Wesley, 2015.
[5] A. Cockburn, *Hexagonal Architecture*, 2005. dirección: https : / / alistair . cockburn . us / hexagonal - architecture